

09-12-2025

Lab - 3

Lab: Scikit-Learn Fundamentals (Google Play Store)

Objective: Transition from manual data cleaning to automated Machine Learning preprocessing using Scikit-Learn.

Prerequisites:

- Ensure you have the `googleplaystore_cleaned.csv` file (from the previous lab) in this folder.

```
In [ ]: import pandas as pd
```

1. Load Preprocessed Data

Instruction: Load the dataset you cleaned in the previous lab. This dataset should already have `Installs`, `Price`, and `Reviews` converted to numbers.

```
In [ ]: df = pd.read_csv("Cleaned_File.csv")
df.head()
```

```
In [ ]: df.shape
```

Intro to Scikit-Learn

What is Scikit-Learn? It is the standard library for Machine Learning in Python. We use it for:

1. **Preprocessing:** Scaling numbers and encoding text.
2. **Modeling:** Training algorithms.
3. **Evaluation:** Checking accuracy.

Task: Import `sklearn` and check the version.

```
In [ ]: import sklearn
```

```
In [ ]: sklearn.__version__
```

3. Train_Test_Split

Concept: We split data to prevent "Overfitting". The model learns from the **Train** set and is tested on the **Test** set.

Task:

1. Define `X` (Features: everything except Rating/App) and `y` (Target: Rating).
2. Split the data (80% Train, 20% Test).

```
In [ ]: from sklearn.model_selection import train_test_split
```

```
In [ ]: X = df.drop(['Rating', 'App'], axis=1)
        y = df['Rating']
```

```
In [ ]: # train_test_split
        X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42, test_size=0.2)
```

```
In [ ]: print(f"X-Training Shape: {X_train.shape}")
        print(f"X-Testing Shape: {X_test.shape}")
```

4. Scaling Numerical Data (StandardScaler)

Concept: `Installs` (Millions) are much larger than `Rating` (1-5). We scale them so the model treats them equally.

Task: Use `StandardScaler` on the numerical columns.

```
In [ ]: from sklearn.preprocessing import StandardScaler
```

```
In [ ]: num_cols = ['Reviews', 'Size', 'Installs', 'Price']
```

```
In [ ]: scaler = StandardScaler()
```

```
In [ ]: X_train_scaled = scaler.fit_transform(X_train[num_cols])
```

```
In [ ]: X_train_scaled
```

5. Encoding Categorical Data

Concept: Models need numbers, not text like "Business" or "Teen".

Method A: Pandas `get_dummies` (Simple)

```
In [ ]: dummies = pd.get_dummies(X_train['Content Rating'])
```

```
In [ ]: dummies.head()
```

Method B: Sklearn OneHotEncoder (Professional)

```
In [ ]: from sklearn.preprocessing import OneHotEncoder
```

```
In [ ]: encoder = OneHotEncoder(handle_unknown='ignore')
```

```
In [ ]: cat_encoded = encoder.fit_transform(X_train[['Category']])
```

```
In [ ]: cat_encoded.shape
```

6. 🚀 The Full Pipeline: ColumnTransformer

Concept: Instead of doing steps 4 and 5 manually, we wrap them in one object.

Task: Create a `ColumnTransformer` that Scales numerical data AND Encodes categorical data at the same time.

```
In [ ]: from sklearn.compose import ColumnTransformer
```

```
In [ ]: numeric_features = ['Reviews', 'Size', 'Installs', 'Price']  
categorical_features = ['Category', 'Content Rating']
```

```
In [ ]: # Create ColumnTransformer  
preprocessor = ColumnTransformer(transformers=  
                                [ ('num', StandardScaler(), numeric_features),  
                                  ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)  
                                ],  
                                remainder='passthrough')  
)
```

```
In [ ]: from sklearn.pipeline import Pipeline
```

```
In [ ]: pipeline = Pipeline(steps=[('preprocessor', preprocessor)])
```

```
In [ ]: from sklearn import set_config
```

```
In [ ]: set_config(display='diagram')  
pipeline
```

```
In [ ]: X = df[numeric_features+categorical_features]
```

```
In [ ]: X_Procsted = pipeline.fit_transform(X)
```

```
In [ ]: X_Procsted.shape
```